Batch-Rotation Wallet Privacy Policy, Implementation Spec

Hilawe Semunegus, policy design by Joel Valenzuela (TheDesertLynx)

Implementation spec for project D2 in dash-ideas-scoping.md. The rationale, the threat model, and the contingency framing (build only if D1's per-payment path proves too costly or slow) live in the scoping document. This is the buildable version, the state machine, the default parameters, and the measurement gate.

Plain words. Keep savings shielded and spend from a small transparent pot that refills itself from the shielded balance. Each refill is a sealed batch that is never co-spent with another, so an observer sees short unlinkable chapters instead of one lifelong wallet. This spec pins down the exact states, the numbers, and the tests that decide whether it ships.

State

The wallet tracks:

- a shielded reserve balance
- zero or more transparent batches, each with a batch id, a set of UTXOs, a creation time, and a status (active, draining, returning, or dead)
- exactly one active batch at a time for spending
- a return queue of leftover UTXOs pending re-shielding
- an encrypted metadata backup of all of the above

Parameters (defaults, all tunable)

- floor and ceiling of the spending band, for example 5 and 10 DASH
- refill amount, randomized around the ceiling, drawn from a distribution shaped like organic shielded-to-transparent exits rather than clustered at a round number
- refill schedule, randomized and proactive, so a refill happens before the floor is hit rather than on hitting it
- return delay, randomized and always longer than the refill lead time
- return piece sizes, randomized
- maximum coexisting dead batches
- batch lifetime cap

State machine

- Refill. On the randomized schedule, if the active batch is near or below the floor, draw a randomized amount from the shielded reserve to a brand-new transparent address. This becomes the new active batch, and the old batch moves to draining.
- Spend. Normal wallet behavior within the active batch, change addresses, combined inputs, single-address invoices. Never spend across batches. A payment larger than the active batch is paid directly from the shielded reserve, bypassing the pot.

- Return. A draining batch's leftovers are re-shielded later, in randomized pieces at randomized times, always slower than the refill so the user is never short of spendable coins. Only one rotation runs at a time.
- Dust. Small remainders in dead batches are swept back one batch at a time, never combined.

Liquidity invariant. Spendable coins never depend on the return queue, because any shortfall is covered by paying directly from the shielded reserve.

The four privacy rules

1. Batches never mix. Coins from different refills are never co-spent.
2. Amounts are randomized, not round, and shaped to blend with organic exits.
3. Refill and return are decoupled in time, so an observer cannot pair "went out here" with "came back there."
4. All broadcasts route over Tor, so network origin does not re-link what the chain rules separated.

Residual risks and the mitigation each imposes on this spec

- Trigger observability. Refill on a randomized schedule ahead of need so no visible dip-then-refill sequence exists.
- Counterparty reuse. Warn on cross-batch reuse of a recipient address, or route repeat counterparties directly from the shielded reserve.
- Wallet fingerprints. Match transaction shape, fee policy, and output ordering to the dominant wallet.
- Light-client lookup. Add private address lookup so the servers a phone wallet queries cannot re-link batches, the same infrastructure the D3 annex needs.
- State recovery. Keep an encrypted metadata backup, plus a safe degraded restore that treats all recovered coins as one sealed batch to be swept through the shielded balance.
- Fee and mempool behavior. Ensure refill, return, and sweep transactions clear under mempool minimums and fee spikes, since fee-driven timing can itself mark rotation traffic.

Measurement gate

Prototype behind a toggle. Ship or kill on these:

- boundary-correlation resistance under simulation at small and realistic adoption
- fee overhead per rotation epoch
- stranded-dust rate
- a restore drill, recover from seed plus metadata backup mid-rotation with no batch merging or fund loss
- zero regressions on invoice and deposit flows

Dependency

The privacy of this policy rests on a healthy, busy shielded pool, since the crowd is the privacy. The reserve is the Orchard shielded balance shipping in Platform v4.0, and where that is not yet available a CoinJoin-mixed reserve can stand in with the same mechanics. Whether this policy is needed at all depends on D1. If per-payment withdrawals directly from the shielded balance prove fast and cheap enough, most users need no pot, so build only if the D1 measurements say otherwise.

Credit

The batch-rotation policy is Joel Valenzuela's design, refined over several rounds. This spec formalizes it.